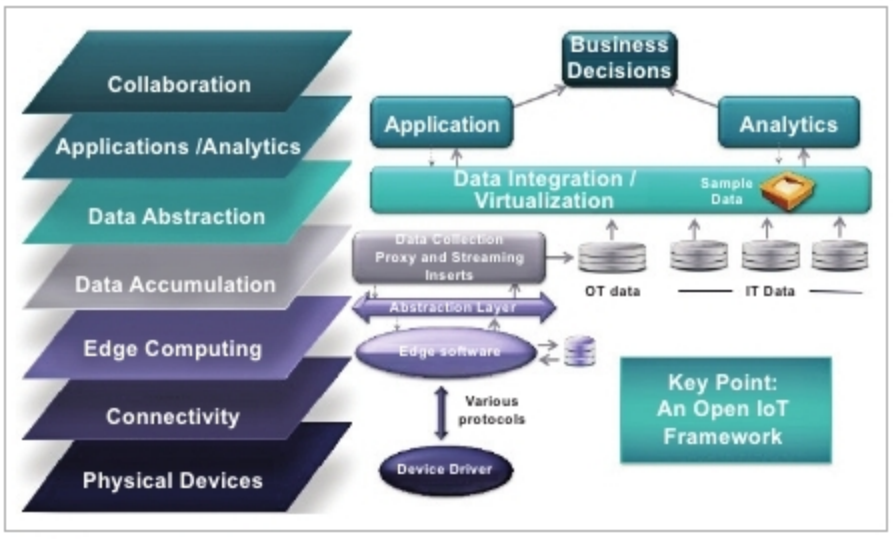


Figure 1: IoT architecture

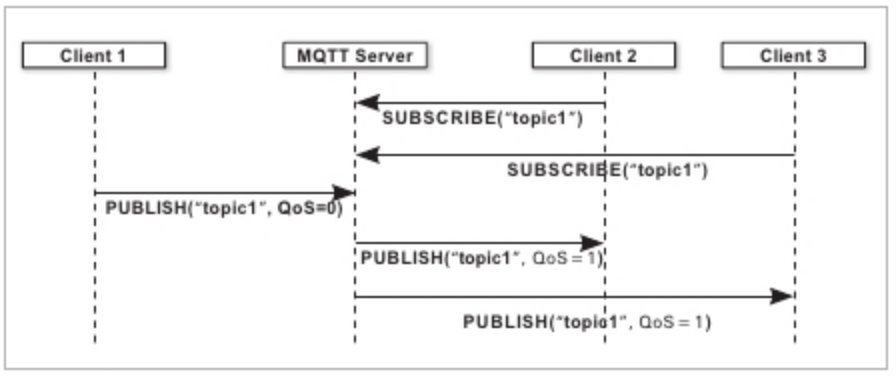


Figure 2: Sequence diagram

low-impact data movement protocol used by a wide variety of IoT objects and operational platforms to communicate over the network. Introduced by IBM in 1999, it provides the connectivity between applications and users at one end, and the network and communications protocols at the other end. Applications making use of MQTT can be developed just by implementing its control packets—connect, publish, subscribe and disconnect.

The MQTT protocol was specifically built to support machine-to-machine (M2M) and IoT applications. Its lightweight design has facilitated a revolution in intercommunication performance. MQTT has thus enabled rapid messaging between the billions of things that are now connected through the Internet.

## The MQTT framework

MQTT has a publish/subscribe architecture, consisting of three main components — the publishers, the subscribers and the brokers. The publishers are the lightweight sensors that connect to the broker to send their data and go back to sleep whenever possible. The subscribers are applications that are interested in a certain topic, or sensory data, so they connect to brokers to be informed whenever new data are received. And the brokers classify sensory data in topics and send them to subscribers interested in those topics only.

The communication is asynchronous, and can also act as a pipe to binary data. Every message published to an address is called a topic. Clients may subscribe to multiple topics. Every client subscribing to a topic receives all the messages published to the topic. In an MQTT skeleton, sensors act as the clients and the server acts as a broker over a TCP layer. The client can either publish or subscribe to a message. An extension of MQTT, secure MQTT (SMQTT), was proposed in order to provide lightweight attribute based encryption.

In Figure 2, client1 wishes to share a particular topic (message) by publishing it through the MQTT server once it is available. Clients 2 and 3 subscribe to (acknowledge) the message. 'QoS=0' means the client doesn't expect any response to the message sent, whereas 'QoS=1' means the message is stored and a response (PUBACK Response) is expected.

## Applications and use cases

Facebook currently uses the MQTT protocol for its messaging app, not only because the protocol conserves battery power during mobile phone-to-phone messaging, but also because, in spite of inconsistent Internet connections across the globe, the protocol enables messages to be delivered efficiently within milliseconds. Most cloud service providers including AWS, Google Cloud, IBM Bluemix and Microsoft Azure also follow the same protocol. MQTT is well suited to applications using M2M and IoT devices, for real-time analytics, preventive maintenance and monitoring in environments such as smart homes, healthcare, logistics, industry and manufacturing.

## Competing protocols

Other transfer protocols under consideration for IoT devices with constrained resources include the Constrained Application Protocol (CoAP), which uses a request/response communication pattern, and the Advanced Message Queuing Protocol (AMQP), which, like MQTT, uses a publish/subscribe communication pattern.

MQTT is a lightweight communications protocol and can be used for many purposes. Imagine your alarm clock knows that your train to work is 15 minutes late and adjusts itself accordingly, or that your coffee maker switches on automatically 15 minutes later to make you a hot cup of coffee before you leave for work. Sounds like the future? Actually, all this is possible even today. Ericsson predicts that in 2020, 50 billion devices will be connected over the Internet. The communication between these objects will be enabled by IPv6 and lightweight communication protocols like MQTT. Our future is in safe hands! END

## References

[1] *https://www.hivemq.com*
[2] *https://internetofthingsagenda.techtarget.com*
[3] *http://www.iri.com*

**By: Neetesh Mehrotra**

The author works at NIIT Technologies as a senior test engineer, and his areas of interest are Java development and automation testing. You can contact him at *mehrotra.neetesh@gmail.com*.